# Lab 07 - Evolutionary robotics

**Author**: Diego Colì - diego.coli@studio.unibo.it

## Idea

The goal is to evolve a neural-network controller capable of performing phototaxis, by using a Genetic Algorithm (GA). The evolutionary process evaluates candidate solutions in simulation and progressively improves them through selection, crossover, mutation, and elitism.

## Controller architecture

The controller (`controller-nn.lua`) is based on `nn.lua` (a simple feed-forward neural network):

- **Inputs**: 24 light sensor readings (`robot.light`).
- **Outputs**: 2 output neurons controlling left and right wheel velocities.
- **Activation function**: sigmoid.
- **Genome encoding**: all network weights and output biases are stored in a vector of floating-point numbers (`GENOME_LENGTH = 50`).

At each control step:

- Light sensor values are collected.
- The neural network computes the outputs.
- Outputs are scaled by `MAX_VELOCITY = 15`.
- The wheel velocities are applied to the robot.

The fitness is computed at the end of the simulation as `fitness = (dmax - d) / dmax`, where `d` is the Euclidean distance from the light source and `dmax` is the maximum possible distance in the arena.

## Genetic Algorithm

The GA (`ga.py`) implements the following components:

- Population initialization: random genomes in [-1,1].
- Fitness evaluation: each individual is evaluated `N_EVAL = 3` times to reduce stochastic effects caused by noise and random initial conditions.
- Selection operator: tournament (roulette wheel as an alternative).
- Crossover: linear interpolation between parent genomes.
- Mutation: Gaussian mutation with mean 0 and standard deviation 1, applied independently to each gene with probability `MUTATION_RATE = 0.1`.
- Replacement strategy: generational replacement.
- Elitism: the best `ELITE_SIZE` individuals are preserved across generations.

## Expected behavior

- Random initial genomes produce mostly chaotic or ineffective behaviors.
- Over generations, evolution discovers weight configurations that steer the robot toward the light.

- Tournament selection generally produces faster convergence due to stronger selective pressure.
- Elitism improves stability by preserving good solutions across generations.
- Using the minimum fitness across evaluations promotes more robust behaviors in noisy conditions.
- High crossover rates increase exploration but may also disrupt good solutions if excessive.

Overall, the evolved controller demonstrates how relatively simple neural networks can generate effective reactive robot behaviors through evolutionary optimization alone.

## Experimental results

Several experiments were performed by varying:

- Crossover probability (`CX_RATE ∈ {0, 0.5, 1}`).
- Selection strategy.
- Elitism.
- Fitness aggregation (mean vs min over multiple evaluations).

Results from 5 replicas were collected and compared through means and medians.

### Crossover rate comparison

| `CX_RATE` | **Median fitness** | **Mean fitness** |
|---|---|---|
| 0.0 | 0.5434 | 0.6076 |
| 0.5 | 0.9056 | 0.6706 |
| 1.0 | 0.6572 | 0.6035 |

The experiments show how crossover influences exploration and convergence speed. Moderate crossover rates generally produced more stable results.

### Elitism vs replacement

| **Strategy** | **Median fitness** | **Mean fitness** |
|---|---|---|
| No elitism | 0.5702 | 0.5944 |
| Elitism (5 individuals) | 0.9364 | 0.8925 |

Elitism improved stability and prevented the loss of high-quality solutions across generations.

### Tournament vs roulette wheel selection

| **Selection** | **Median fitness** | **Mean fitness** |
|---|---|---|
| Tournament | 0.7680 | 0.7738 |
| Roulette wheel (proportional) | 0.4790 | 0.6153 |

Tournament selection usually achieved faster convergence due to stronger selective pressure, while roulette wheel selection maintained higher diversity.